

Không gian truy xuất bằng chứng

cho QA đa bước và tìm kiếm siêu dữ liệu ngữ nghĩa

Báo cáo kỹ thuật nháp cho mentor

Cập nhật: 30/06/2026

- Luận điểm** Hệ thống tập trung vào tầng truy xuất: tìm đúng bằng chứng, mở rộng lên toàn bộ corpus, đánh giá bằng metric phù hợp cho QA đa bước, và làm kết quả truy xuất có thể giải thích được.
- Phạm vi** Không trình bày như nhật ký sprint. Báo cáo đi theo bài toán, kiến trúc, phương pháp, đánh giá, tìm kiếm siêu dữ liệu ngữ nghĩa, giới hạn và bước tiếp theo.

Mục lục

1	Tóm tắt	3
2	Định nghĩa bài toán	3
3	Kiến trúc: tách luồng xử lý offline và luồng truy vấn online	3
3.1	Luồng ingest/xử lý offline	3
3.2	Luồng retrieval online	4
4	Các phương pháp retrieval	4
5	Các thí nghiệm ablation	5
5.1	Thí nghiệm index: BM25 có title-aware	5
5.2	Thí nghiệm gộp hạng: RRF so với cross-encoder reranker	5
5.3	Thí nghiệm sinh ứng viên: truy xuất bridge-aware	6
5.4	Thí nghiệm siêu tham số: tinh chỉnh truy xuất bridge-aware	6
5.5	Thí nghiệm độ bền truy vấn: paraphrase	6
5.6	Thí nghiệm họ retrieval: BM25 so với dense và hybrid	6
5.7	Thí nghiệm filtered hybrid: hybrid rộng so với hybrid có ràng buộc metadata	7
5.8	Thí nghiệm cross-dataset: VimQA BM25 so với dense và hybrid	7
5.9	Thí nghiệm metadata filter: thu hẹp không gian ứng viên	7
5.10	Thí nghiệm chiến lược tài liệu: mức tài liệu so với mức đoạn	8
5.11	Thí nghiệm backend dense: Elasticsearch dense so với TurboVec và chỉ BM25	8
5.12	Thí nghiệm quy mô cũ: nano/100k và truy xuất lập Elasticsearch	8
6	Độ trễ và đánh đổi thời gian chạy	9
7	Đánh giá đa bước và chặn đoán lỗi	9
8	Độ bền truy vấn và thí nghiệm cross-dataset	10
8.1	Độ bền với paraphrase	10
8.2	VimQA Tiếng Việt	10
9	Tìm kiếm metadata ngữ nghĩa	11
10	Thời gian chạy theo dataset và lớp giải thích	11
11	Luồng demo đề xuất	12
12	Khẳng định an toàn, điều không khẳng định và bước tiếp theo	12
12.1	Khẳng định an toàn	12
12.2	Điều không khẳng định	12
12.3	Bước tiếp theo	13
13	Câu Hỏi Mentor Có Thể Hỏi	13
A	Phụ lục cấu hình	13
A.1	Hồ sơ dataset trong runtime	14
A.2	Cấu hình ingest và artifact	14
A.3	Cấu hình retrieval và benchmark	15
A.4	Cấu hình runtime, API và cache	15
A.5	Cấu hình semantic metadata	16
A.6	Lệnh reproduce tối thiểu	16

A.7 Cấu hình render báo cáo

16

1 Tóm tắt

Dự án xây dựng một không gian truy xuất bằng chứng cho bài toán hỏi đáp đa bước và tìm kiếm siêu dữ liệu ngữ nghĩa. Trọng tâm của hệ thống không phải sinh câu trả lời cuối cùng, mà là trả về các tài liệu/ngữ cảnh có thể dùng làm bằng chứng cho bước suy luận phía sau.

Ba điểm chính cần mentor nắm:

- **Mở rộng toàn bộ tập dữ liệu:** hệ thống chạy trên toàn bộ tập HotpotQA với 5,233,329 tài liệu, dùng Elasticsearch cho BM25/kho tài liệu và TurboVec cho truy xuất dense nén.
- **Đánh giá đúng bài toán đa bước:** metric quan trọng nhất là `full_support_recall@10`, vì một truy vấn HotpotQA có thể cần đủ nhiều tài liệu hỗ trợ.
- **Tìm kiếm siêu dữ liệu ngữ nghĩa có thể giải thích:** truy vấn tự nhiên có thể được tách thành `effective_query` và `metadata_filters`, nhưng bộ phân tích chỉ chạy khi được bật thủ công để không làm nhiều truy vấn gốc.

Kết quả mới nhất đáng nhấn mạnh là hướng `tv_bridge_title_entities_rrf`. Trong pilot 200 truy vấn, phương pháp này nâng `full_support@10` từ 0.545 của `tv_hybrid` lên 0.620. Cấu hình tuning `beam1_terms6` giữ nguyên 0.620 và giảm độ trễ p95 xuống khoảng 1225 ms.

2 Định nghĩa bài toán

Bài toán được định nghĩa là **truy xuất bằng chứng**, không phải sinh câu trả lời. Với mỗi câu hỏi hoặc truy vấn tự nhiên, hệ thống trả về top-k tài liệu có khả năng làm bằng chứng. Với HotpotQA, một câu hỏi thường cần nhiều tài liệu hỗ trợ. Nếu hệ thống chỉ tìm được một tài liệu hỗ trợ nhưng thiếu tài liệu còn lại, bước suy luận phía sau vẫn có thể thất bại.

Vì vậy, ngoài `recall@k`, hệ thống dùng `full_support_recall@k`. Metric này đo tỷ lệ truy vấn mà top-k kết quả chứa đủ toàn bộ tài liệu hỗ trợ.

Trong truy xuất đa bước, câu hỏi không chỉ là top-k có tài liệu liên quan hay không, mà là top-k có đủ bộ tài liệu hỗ trợ hay không.

3 Kiến trúc: tách luồng xử lý offline và luồng truy vấn online

Kiến trúc nên được trình bày bằng hai luồng riêng. Cách tách này làm rõ phần nào chạy trước để xây artifact, phần nào chạy tại thời điểm truy vấn để phục vụ người dùng.

3.1 Luồng ingest/xử lý offline

Luồng ingest/xử lý chạy trước thời gian chạy. Mục tiêu là biến corpus thô thành các artifact ổn định có thể tìm kiếm được.

```
Tập dữ liệu gốc
-> chuẩn hóa tài liệu
-> gán numeric_id ổn định
-> sinh siêu dữ liệu tổng hợp
-> chia shard JSONL staging
-> xây index BM25 trên Elasticsearch
-> sinh shard embedding
-> xây index dense TurboVec 4-bit
-> kiểm tra số lượng tài liệu và artifact
```

Các artifact chính:

Artifact	Vai trò
Shard JSONL staging	Chuẩn hóa corpus, chia shard, giữ <code>numeric_id</code> ổn định.
Index Elasticsearch	Truy xuất lexical bằng BM25, lọc siêu dữ liệu, hydrate tài liệu.
Shard embedding	Vector hóa nội dung bằng BGE-small hoặc model theo từng dataset.
TurboVec .tvim	Truy xuất dense nén 4-bit cho toàn bộ corpus.
Artifact siêu dữ liệu	Siêu dữ liệu tổng hợp để demo truy xuất có nhận biết metadata.

3.2 Luồng retrieval online

Luồng retrieval chạy khi người dùng nhập truy vấn. Luồng này không xử lý lại corpus, mà dùng các index đã xây sẵn.

```

Truy vấn người dùng
-> bộ phân tích siêu dữ liệu ngữ nghĩa tùy chọn
-> effective_query + metadata_filters
-> truy xuất BM25 từ Elasticsearch
-> truy xuất dense từ TurboVec
-> hydrate kết quả dense qua Elasticsearch numeric_id
-> gộp hạng bằng RRF hoặc bridge-aware
-> overlay support + highlight + chip đã parse
-> kết quả trên dashboard React

```

Điểm thiết kế quan trọng là Elasticsearch và TurboVec không thay thế nhau. Elasticsearch giữ vai trò truy xuất lexical, lọc và kho tài liệu. TurboVec giữ vai trò truy xuất dense nén ở quy mô toàn bộ corpus. Hai backend được nối bằng `numeric_id`; tầng ứng dụng hydrate kết quả dense và gộp hạng bằng RRF hoặc chiến lược bridge-aware.

4 Các phương pháp retrieval

Phương pháp	Vai trò
<code>es_bm25</code>	Mốc so sánh lexical nhanh, mạnh khi truy vấn có độ trùng entity.
<code>tv_dense</code>	Truy xuất ngữ nghĩa bằng TurboVec, hữu ích khi truy vấn ít trùng từ khóa.
<code>tv_hybrid</code>	BM25 + ứng viên dense, gộp hạng bằng Reciprocal Rank Fusion.
<code>tv_filtered_hybrid</code>	BM25 prefilter + truy xuất dense bị ràng buộc bởi metadata/filter.
<code>tv_bridge_title_entities_rrf</code>	Truy xuất bridge-aware cho tài liệu hỗ trợ thứ hai trong QA đa bước.

Kết quả pilot 200 truy vấn dev trên toàn bộ HotpotQA corpus:

Phương pháp	Full-support@10	Độ trễ p95	Diễn giải
es_bm25	0.365	359.63 ms	Mốc so sánh lexical nhanh, nhưng thiếu nhiều bộ support đầy đủ.
tv_dense	0.515	868.00 ms	Đường dense cải thiện rõ trên toàn bộ corpus.
tv_hybrid	0.545	2061–3089 ms	Mốc so sánh hybrid ưu tiên chất lượng, tùy lần chạy/cấu hình.
tv_bridge_title_entities_rrf	0.620	1224.99 ms	Kết quả pilot tốt nhất sau tuning beam1_terms6.

Kết luận kỹ thuật: không có phương pháp thắng tuyệt đối trong mọi hoàn cảnh. BM25 tốt về độ trễ, dense tốt cho độ bền ngữ nghĩa, hybrid cân bằng lexical/dense, còn truy xuất bridge-aware đánh trực tiếp vào lỗi thiếu tài liệu hỗ trợ trong truy xuất đa bước.

5 Các thí nghiệm ablation

Các thí nghiệm ablation trong dự án không chỉ so sánh phương pháp nào cao điểm hơn. Mục tiêu chính là xác định phần tăng chất lượng đến từ đâu: index lexical, truy xuất dense, gộp hạng RRF, reranking, sinh ứng viên bridge-aware, hay độ bền trước paraphrase.

5.1 Thí nghiệm index: BM25 có title-aware

BM25 có nhận biết title tăng trọng số các trường gần với title/entity như `title_exact`, `lead_sentence` và `title_repeat_content`. Kết quả cho thấy các metric lexical tăng nhẹ, nhưng metric chính của HotpotQA không đổi.

Phương pháp	Recall@10	MRR@10	nDCG@10	Full-support@10
es_bm25	0.6025	0.7108	0.5727	0.3650
es_bm25_title	0.6050	0.7159	0.5786	0.3650

Diễn giải: index có nhận biết title là ablation sạch và rẻ, nhưng không giải quyết lỗi chính của HotpotQA là thiếu tài liệu hỗ trợ thứ hai. Vì vậy phương pháp này được giữ làm benchmark ablation, không làm mặc định.

5.2 Thí nghiệm gộp hạng: RRF so với cross-encoder reranker

Thí nghiệm reranker dùng cross-encoder/ms-marco-MiniLM-L-6-v2 với ngân sách 100 ứng viên. Kết quả cho thấy reranker cải thiện một số rank-sensitive metrics, nhưng không cải thiện `full_support@10`.

Phương pháp	Full-support@10	Recall@10	MRR@10	nDCG@10	Độ trễ p95
tv_hybrid	0.5450	0.7500	0.8691	0.7291	2061.4370 ms
tv_hybrid_rerank	0.5450	0.7550	0.9268	0.7464	1304.2816 ms

Dịch chuyển theo cặp trên 200 truy vấn:

- Chỉ RRF thành công: 14.
- Chỉ reranker thành công: 14.
- Cả hai cùng thành công: 95.
- Cả hai cùng thất bại: 77.

- Net win của reranker: 0.

Diễn giải: chưa có bằng chứng đủ mạnh để nói reranker là hướng chính. Nếu full-support không tăng, vẫn đề không chỉ nằm ở việc sắp xếp lại ứng viên. Cần cải thiện bước sinh ứng viên cho tài liệu hỗ trợ bị thiếu.

5.3 Thí nghiệm sinh ứng viên: truy xuất bridge-aware

Truy xuất bridge-aware được thiết kế để đánh vào lỗi lớn nhất: truy vấn tìm được một tài liệu hỗ trợ nhưng thiếu tài liệu hỗ trợ còn lại. So với `tv_hybrid` chạy một lượt, phương pháp bridge-aware tạo truy vấn bước hai từ tín hiệu `title/entity/lead-sentence` của các ứng viên bước đầu.

Phương pháp	Full-support@10	Recall@10	nDCG@10	Độ trễ p95
<code>tv_hybrid</code>	0.5450	0.7500	0.7291	1146.5764 ms
<code>tv_two_hop_bridge_rrf</code>	0.5600	0.7450	0.6999	2773.5883 ms
<code>tv_bridge_title_entities_rrf</code>	0.6200	0.7850	0.7398	2670.3591 ms

Chẩn đoán cho thấy phương pháp bridge title/entities chuyển thêm 15/200 truy vấn thành full-support thành công và giảm lỗi partial-support từ 82 xuống 66. Đây là ablation đầu tiên tạo mức tăng rõ ràng trên metric chính.

5.4 Thí nghiệm siêu tham số: tinh chỉnh truy xuất bridge-aware

Sau khi truy xuất bridge-aware tạo mức tăng, lưới tinh chỉnh kiểm tra liệu beam rộng có thật sự cần thiết không. Kết quả tốt nhất cho điểm vận hành hiện tại là `beam1_terms6`.

Cấu hình	Beam	Số terms	Full-support@10	nDCG@10	Độ trễ p95
<code>beam1_terms6</code>	1	6	0.6200	0.7382	1224.9911 ms
<code>beam2_terms4</code>	2	4	0.6100	0.7430	1758.8852 ms
<code>beam2_terms6</code>	2	6	0.6200	0.7423	2593.4644 ms
<code>beam2_terms8</code>	2	8	0.6200	0.7399	1827.7998 ms

Diễn giải: beam rộng hơn không tăng full-support trong pilot này. `beam1_terms6` giữ 0.6200 full-support và giảm mạnh độ trễ p95, nên là cấu hình tiếp theo cần được đánh giá lớn hơn.

5.5 Thí nghiệm độ bền truy vấn: paraphrase

Benchmark paraphrase gây áp lực lên truy xuất bằng cách đổi truy vấn ở ba mức: mild, strong và lexical-strong. Mild/strong chủ yếu kiểm tra đảo trật tự câu, còn lexical-strong mới thật sự thay từ nội dung để gây áp lực lên truy xuất lexical.

Phương pháp	Gốc	Mild	Strong	Lexical strong
<code>es_bm25</code>	0.365	0.365	0.375	0.340
<code>tv_dense</code>	0.515	0.515	0.515	0.495
<code>tv_hybrid</code>	0.535	0.515	0.515	0.480
<code>tv_filtered_hybrid</code>	0.430	0.435	0.440	0.395

Diễn giải: `tv_dense` ổn định nhất khi độ trùng lexical bị phá (-0.020 full-support), còn `tv_hybrid` tốt trên truy vấn gốc nhưng giảm mạnh hơn ở lexical-strong (-0.055). Đây là lý do đường dense vẫn cần thiết, dù BM25/hybrid có thể tốt hơn ở truy vấn gốc.

5.6 Thí nghiệm họ retrieval: BM25 so với dense và hybrid

Đây là ablation nền tảng nhất của HotpotQA toàn corpus: chỉ lexical, chỉ dense, hybrid RRF, và filtered hybrid. Nó trả lời câu hỏi “có cần đường dense không, và gộp hạng có đáng giá không?”.

Phương pháp	Recall@10	MRR@10	nDCG@10	Full-support@10
es_bm25	0.6025	0.7108	0.5727	0.3650
tv_dense	0.7225	0.8472	0.7082	0.5150
tv_hybrid	0.7500	0.8681	0.7286	0.5450
tv_filtered_hybrid	0.6800	0.8225	0.6735	0.4550

Diễn giải: truy xuất dense bằng TurboVec tạo mức tăng lớn so với BM25 trên toàn bộ corpus (0.3650 → 0.5150 full-support). Hybrid RRF tiếp tục cải thiện lên 0.5450, nên dense và gộp hạng đều có bằng chứng. Filtered hybrid thấp hơn vì BM25 prefilter có thể loại mất tài liệu hỗ trợ thứ hai.

5.7 Thí nghiệm filtered hybrid: hybrid rộng so với hybrid có ràng buộc metadata

tv_filtered_hybrid là thí nghiệm ablation giữa hybrid rộng ưu tiên chất lượng và đường có ràng buộc cứng từ allowlist BM25/filter. Đây là phương pháp quan trọng cho tìm kiếm metadata, nhưng không nên đọc như phương pháp chất lượng mặc định của HotpotQA.

Phương pháp	Full-support@10	Recall@10	nDCG@10	Độ trễ p95
tv_hybrid	0.5450	0.7500	0.7286	3089.2229 ms
tv_filtered_hybrid	0.4550	0.6800	0.6735	1953.6118 ms

Diễn giải: filtered hybrid giảm độ trễ so với hybrid rộng, nhưng mất 0.090 điểm tuyệt đối full-support. Đánh đổi này hợp lý cho ràng buộc metadata, không hợp lý để thay mặc định HotpotQA khi mục tiêu là phủ bằng chứng tối đa.

5.8 Thí nghiệm cross-dataset: VimQA BM25 so với dense và hybrid

VimQA kiểm tra giả định “dense/hybrid luôn thắng BM25” trên proxy truy xuất tiếng Việt. Dataset này có 3,623 tài liệu và 9,044 truy vấn/qrels.

Phương pháp	Recall@10	MRR@10	nDCG@10	Độ trễ p95	Quyết định
es_bm25	0.9627	0.8606	0.8859	84.4191 ms	Mặc định tốt nhất.
es_dense	0.8716	0.7272	0.7625	115.0396 ms	Dense không thắng BM25.
es_hybrid	0.9644	0.8277	0.8609	206.3031 ms	Recall nhìn hơn nhưng xếp hạng/độ trễ kém hơn.

Diễn giải: đặc tính dataset quyết định phương pháp. Trên VimQA, độ trùng lexical cao và mỗi truy vấn có một gold context, nên BM25 là phương án tốt nhất khi xét cả xếp hạng. Đây là phản ví dụ quan trọng chống lại việc nói quá về dense/hybrid.

5.9 Thí nghiệm metadata filter: thu hẹp không gian ứng viên

Ablation metadata dùng metadata HotpotQA tổng hợp để đo filter làm hẹp không gian ứng viên thế nào. Đây không phải benchmark chất lượng BEIR, mà là bằng chứng về hành vi sản phẩm/tìm kiếm.

Kịch bản	Bộ lọc	Số tài liệu khớp	Mức thu hẹp
Chỉ nội dung	không có	5,233,329	0.0000%
Lọc tác giả	author=Nguyen An	40,886	99.2187%
Lọc ngày tạo	created_at trong tháng 01/2024	222,239	95.7534%
Lọc ngày sửa	modified_at 10–20/01	60,589	98.8422%
Tác giả + ngày tạo	author + created_at	1,793	99.9657%

Diễn giải: metadata có cấu trúc giúp corpus 5.23M tài liệu dễ kiểm tra hơn. Kịch bản tác giả + ngày tạo thu hẹp không gian ứng viên 99.9657% trước bước xếp hạng theo nội dung. Khẳng định an toàn: lọc metadata hoạt động như một cơ chế demo/tìm kiếm có kiểm soát. Không khẳng định: metadata tổng hợp là tác giả thật của HotpotQA hoặc metadata meeting production.

5.10 Thí nghiệm chiến lược tài liệu: mức tài liệu so với mức đoạn

EDA về chiến lược tài liệu kiểm tra liệu chia đoạn rộng có nên là bước nâng chất lượng tiếp theo ở tầng ingest hay không. Kết quả không ủng hộ việc chọn chia đoạn rộng làm bước đầu tiên.

Chỉ số	p50	p90	p95	p99
Số token nội dung	39	90	111	160
Số câu/tài liệu	2	5	6	8

Chiến lược đoạn	Số đơn vị truy xuất ước tính	Tăng so với mức tài liệu
80 tokens, stride 40	6,217,195	+18.8%
120 tokens, stride 60	5,458,362	+4.3%
160 tokens, stride 80	5,290,896	+1.1%

Diễn giải: tài liệu HotpotQA vốn đã ngắn, nên index đoạn toàn diện phần lớn tái tạo lại index ở mức tài liệu nhưng thêm độ phức tạp gom parent. Hướng tốt hơn trước mắt là title/entity và truy xuất bridge-aware, không phải chia passage rộng.

5.11 Thí nghiệm backend dense: Elasticsearch dense so với TurboVec và chỉ BM25

Sprint 3 cũng có ablation về kiến trúc, không chỉ là bảng metric. Các phương án được cân nhắc:

Phương án	Quyết định
Elasticsearch toàn corpus	Loại cho quy mô local 5.23M tài liệu vì overhead RAM/HNSW quá rủi ro.
Truy xuất toàn quy mô chỉ bằng BM25	Loại làm mục tiêu cuối vì so sánh dense/hybrid là mục tiêu lỗi.
Lượng tử hóa TurboVec 2-bit trước	Loại ở bước đầu; BGE-small 384 dim với lượng tử hóa 4-bit là mốc so sánh an toàn hơn.
TurboVec 4-bit + ES BM25/kho tài liệu	Chọn: index dense độc lập, nén, và join ngược về ES bằng numeric_id.

Diễn giải: ablation này giải thích vì sao hệ thống có hai backend truy xuất. TurboVec không thay Elasticsearch; nó thay đường dense vector toàn corpus, còn Elasticsearch vẫn giữ BM25, lọc và kho hydrate tài liệu.

5.12 Thí nghiệm quy mô cũ: nano/100k và truy xuất lặp Elasticsearch

Các thí nghiệm sớm từng so sánh Elasticsearch dense/hybrid và truy xuất lặp ở quy mô nano/100k. Chúng hữu ích như lịch sử thử nghiệm, nhưng không nên dùng làm khẳng định chính vì thời gian chạy hiện tại đã chuyển sang BM25 + TurboVec trên toàn bộ corpus.

Thí nghiệm cũ	Cách trình bày
Nano/100k BM25 so với ES dense/hybrid	Bảng chứng khả thi ban đầu, đã được thay bằng artifact toàn corpus 5.23M tài liệu.
ES hybrid/title/sentence/fast	Loại khỏi đường đang chạy trên toàn corpus vì phụ thuộc các trường ES dense cũ.
Paraphrase syn020/syn040/syn060	Đã được thay bằng benchmark 200 truy vấn mild/strong/lexical-strong có validation.

Diễn giải: chỉ nhắc các thí nghiệm này nếu mentor hỏi lịch sử dự án. Với khẳng định chính, dùng các ablation toàn corpus và các ablation Sprint 4/5 đã được validate ở trên.

6 Độ trễ và đánh đổi thời gian chạy

Độ trễ trong báo cáo này là số đo cục bộ trên laptop Windows khoảng 16 GB RAM, không phải SLA cho môi trường thật. Vai trò của các số độ trễ là giúp giải thích đánh đổi giữa chất lượng, chiến lược truy xuất và độ phức tạp thời gian chạy.

Đường chạy	Vai trò chất lượng	Độ trễ p95	Đánh đổi
es_bm25	Mốc so sánh lexical nhanh	359.6319 ms	Nhanh nhất, nhưng full-support thấp.
tv_dense	Truy xuất ngữ nghĩa	868.0033 ms	Cải thiện full-support, thêm chi phí embedding/tìm kiếm vector.
tv_hybrid	Mốc so sánh chất lượng rộng	2061–3089 ms	Gộp BM25+dense tốt hơn, nhưng tốn hydrate/gộp hạng và đường dense.
tv_filtered_hybrid	Đường metadata/bộ lọc	1953.6118 ms	Nhanh hơn hybrid rộng trong pilot, nhưng mất recall/full-support.
tv_bridge_title_entities_rrf, beam1_terms6	Pilot HotpotQA tốt nhất	1224.9911 ms	Giữ full-support 0.6200 sau tuning, nhưng phương pháp phức tạp và chưa đưa làm mặc định.
VimQA es_bm25	Mặc định tiếng Việt	84.4191 ms	Đặc tính dataset có độ trùng lexical cao, nên BM25 vừa nhanh vừa xếp hạng tốt.

Phân rã độ trễ theo kiến trúc:

- **Đường BM25** chủ yếu tốn truy vấn lexical trên Elasticsearch và lấy tài liệu.
- **Đường dense** thêm embedding cho truy vấn, tìm kiếm vector bằng TurboVec, rồi hydrate bằng Elasticsearch qua `numeric_id`.
- **Đường hybrid** chạy cả BM25 và ứng viên dense, sau đó gộp hạng bằng RRF.
- **Đường filtered hybrid** dùng allowlist BM25/bộ lọc để giảm không gian tìm kiếm, nhưng nếu allowlist thiếu tài liệu hỗ trợ thì full-support giảm.
- **Truy xuất bridge-aware** thêm truy xuất bước hai để tìm tài liệu hỗ trợ bị thiếu; tuning giảm beam/terms để giữ gain nhưng hạ p95.

Kết luận thời gian chạy: BM25 là đường dự phòng/demo khi cần phản hồi nhanh; `tv_hybrid` là mốc so sánh chất lượng đơn giản; `tv_bridge_title_entities_rrf` là ứng viên tốt nhất cho chất lượng HotpotQA sau tuning, nhưng cần benchmark lớn hơn trước khi làm mặc định. Khi trình bày, không nên khẳng định độ trễ môi trường thật cho bất kỳ phương pháp nào.

7 Đánh giá đa bước và chẩn đoán lỗi

HotpotQA cần đánh giá bằng metric phản ánh đủ bộ bằng chứng. Hệ thống dùng:

- **recall@10**: top-10 có tài liệu liên quan hay không.
- **mrr@10**: tài liệu liên quan đầu tiên đứng cao tới đâu.
- **ndcg@10**: chất lượng xếp hạng theo độ liên quan.
- **full_support_recall@10**: top-10 có đủ toàn bộ tài liệu hỗ trợ hay không.

Chẩn đoán xếp hạng chia lỗi thành các nhóm:

Nhóm lỗi	Ý nghĩa
Thiếu ứng viên	Tài liệu hỗ trợ không vào tập ứng viên. Reranker không thể cứu case này.
Chỉ có một phần support	Có một phần support nhưng chưa đủ toàn bộ bộ bằng chứng.
Ứng viên xếp thấp	Tài liệu hỗ trợ có trong ứng viên nhưng bị xếp dưới top-k. Đây là nhóm reranker có thể giúp.
Thành công	Đủ tài liệu hỗ trợ trong top-k.

Ablation reranker top-100 cho thấy **tv_hybrid_rerank** không tạo thắng lợi ròng so với RRF trên pilot 200 truy vấn: cả hai đều đạt **full_support@10=0.545**, với 14 truy vấn chỉ RRF thành công và 14 truy vấn chỉ reranker thành công. Vì vậy hướng cải thiện hợp lý hơn hiện tại là sinh ứng viên/truy xuất tài liệu hỗ trợ thứ hai, dẫn tới **tv_bridge_title_entities_rrf**.

8 Độ bền truy vấn và thí nghiệm cross-dataset

8.1 Độ bền với paraphrase

Benchmark paraphrase kiểm tra khi độ trùng lexical bị thay đổi. Kết quả profile lexical-strong:

Phương pháp	Full-support@10 gốc	Full-support@10 lexical-strong	Chênh lệch
es_bm25	0.365	0.340	-0.025
tv_dense	0.515	0.495	-0.020
tv_hybrid	0.535	0.480	-0.055
tv_filtered_hybrid	0.430	0.395	-0.035

Điều rút ra: truy xuất dense ổn định hơn khi độ trùng lexical bị phá. Hybrid vẫn tốt trên truy vấn gốc, nhưng còn khoảng cách về độ bền cần tiếp tục cải thiện bằng gộp hạng, truy xuất bridge-aware hoặc benchmark sâu hơn.

8.2 VimQA Tiếng Việt

VimQA được dùng như proxy truy xuất tiếng Việt:

- question -> truy vấn
- unique context -> tài liệu
- question-context mapping -> qrel
- answer -> metadata tham khảo, không phải mục tiêu truy xuất

Đặc tính dataset:

Thành phần	Số lượng
Tài liệu	3,623
Truy vấn	9,044
Qrels	9,044

Phương pháp	Recall@10	MRR@10	nDCG@10	Độ trễ p95	Kết luận
es_bm25	0.9627	0.8606	0.8859	84.42 ms	Mặc định tốt nhất.
es_dense	0.8716	0.7272	0.7625	115.04 ms	Dense không thắng BM25.
es_hybrid	0.9644	0.8277	0.8609	206.30 ms	Recall nhỉnh hơn, xếp hạng/độ trễ kém BM25.

Kết luận: không thể giả định dense hoặc hybrid luôn thắng BM25. Phương pháp tốt nhất phụ thuộc đặc tính của dataset.

9 Tìm kiếm metadata ngữ nghĩa

Tìm kiếm metadata ngữ nghĩa trả lời feedback rằng người dùng không nên phải nhập truy vấn nội dung và form bộ lọc riêng biệt. Hệ thống cho phép một truy vấn tự nhiên chứa cả ý định nội dung và ràng buộc metadata.

Ví dụ truy vấn tiếng Việt:

tài liệu về lịch sử Việt Nam của Nguyen An trước 31/01/2024

Bộ phân tích tách thành:

```
original_query = tài liệu về lịch sử Việt Nam của Nguyen An trước 31/01/2024
effective_query = lịch sử Việt Nam
metadata_filters.author = Nguyen An
metadata_filters.created_at_to = 2024-01-31
```

Thiết kế quan trọng:

- Bộ phân tích chỉ chạy khi `semantic_metadata=true`.
- Bộ phân tích rule-based được dùng trước để dễ kiểm soát và test.
- `original_query` được giữ cho hiển thị/history/support lookup.
- `effective_query` được dùng cho truy xuất/highlighting.
- Bộ lọc nhập tay override bộ lọc đã parse để điều khiển rõ ràng từ UI vẫn là nguồn quyết định cuối.
- Metadata không được append vào embedding text để tránh làm nhiễu vector nội dung.

Giới hạn cần nói rõ: tìm kiếm metadata ngữ nghĩa hiện là đường đã cài đặt và có đánh giá smoke/protocol, nhưng chưa đủ lớn để khẳng định bộ phân tích semantic đạt chất lượng sản phẩm thực tế.

10 Thời gian chạy theo dataset và lớp giải thích

Thời gian chạy được thiết kế theo dataset: một API/UI phục vụ nhiều dataset, mỗi dataset có profile riêng về index, phương pháp, phương pháp mặc định, model embedding, metric chính và metadata support.

Dạng endpoint:

```
GET /datasets
GET /datasets/{dataset_id}/stats
GET /datasets/{dataset_id}/queries
GET /datasets/{dataset_id}/benchmarks
GET /datasets/{dataset_id}/embedding-health
POST /datasets/{dataset_id}/search
```

Lớp giải thích gồm:

Lớp	Mục đích
Highlight từ khóa truy vấn	Cho thấy kết quả khớp truy vấn ở đoạn nào.
Overlay support	Cho thấy kết quả nào là gold support hit.
Tóm tắt gold support	Cho biết tài liệu hỗ trợ nào tìm được/chưa tìm được.
Chip metadata đã parse	Cho thấy parser hiểu metadata trong truy vấn thế nào.
Chẩn đoán xếp hạng	Cho biết lỗi nằm ở sinh ứng viên hay xếp hạng.

11 Luồng demo đề xuất

Demo nên minh họa kiến trúc và bằng chứng, không đi theo sprint.

1. Mở dashboard, chỉ selector dataset.
2. Chọn HotpotQA.
3. Chạy một truy vấn bằng `tv_hybrid` hoặc `tv_bridge_title_entities_rrf`.
4. Chỉ phần tóm tắt Gold Support và badge Support Hit.
5. Chỉ các từ khóa truy vấn được highlight trong title/snippet.
6. Bật chế độ Semantic Metadata.
7. Chạy truy vấn: tài liệu về lịch sử Việt Nam của Nguyen An trước 31/01/2024.
8. Chỉ các chip đã parse: Content, Author, Created before.
9. Chuyển sang VimQA.
10. Mở trang benchmark và giải thích vì sao BM25 là mặc định cho VimQA.

12 Khẳng định an toàn, điều không khẳng định và bước tiếp theo

12.1 Khẳng định an toàn

- Hệ thống hỗ trợ truy xuất toàn bộ HotpotQA corpus trên 5,233,329 tài liệu.
- Hệ thống có BM25, TurboVec dense, hybrid RRF, metadata-filtered hybrid và truy xuất bridge-aware.
- `tv_bridge_title_entities_rrf` đạt `full_support@10=0.620` trong pilot 200 truy vấn.
- Hệ thống có dataset-first API/UI cho HotpotQA và VimQA.
- Proxy truy xuất VimQA đã được stage và benchmark trên 9,044 truy vấn.
- Đường Parser/API/UI cho semantic metadata đã được cài đặt và bật theo lựa chọn.

12.2 Điều không khẳng định

- Không khẳng định benchmark HotpotQA có thể so trực tiếp với paper.
- Không khẳng định chất lượng production cho meeting search.
- Không khẳng định độ trễ môi trường thật.
- Không khẳng định tìm kiếm metadata ngữ nghĩa đã được đánh giá đầy đủ.
- Không khẳng định VimQA là benchmark BEIR-style native.

12.3 Bước tiếp theo

1. Chạy benchmark lớn hơn cho truy xuất bridge-aware trên HotpotQA dev/test.
2. Tạo benchmark semantic metadata lớn hơn từ metadata tổng hợp.
3. Định nghĩa schema metadata thật cho meeting search: speaker, participants, timestamp, meeting title, agenda, action items.
4. Gom lại validation command và kiểm tra số lượng artifact.
5. Quyết định phương pháp thời gian chạy mặc định sau khi có bằng chứng lớn hơn pilot 200 truy vấn.

13 Câu Hỏi Mentor Có Thể Hỏi

Vì sao dùng TurboVec?

Vì toàn bộ HotpotQA corpus có 5.23M tài liệu. Elasticsearch rất phù hợp cho BM25 và kho tài liệu, nhưng truy xuất dense vector toàn quy mô có thể nặng trên phần cứng local. TurboVec cho phép truy xuất dense nén, rồi hydrate kết quả qua Elasticsearch bằng `numeric_id`.

Vì sao không chỉ thêm reranker?

Reranker chỉ giúp nếu tài liệu liên quan đã vào tập ứng viên nhưng bị xếp hạng thấp. Ablation reranker top-100 cho thấy net full-support win bằng 0 trên pilot 200 truy vấn. Vì vậy hướng hiệu quả hơn hiện tại là cải thiện sinh ứng viên/truy xuất tài liệu hỗ trợ thứ hai.

Metadata có phải metadata thật không?

Chưa. Metadata của HotpotQA và VimQA hiện là tổng hợp để demo cơ chế truy xuất có nhận biết metadata. Nếu chuyển sang meeting search thật, cần schema thật như speaker, participants, timestamp, agenda và action items.

Truy xuất bridge-aware chứng minh điều gì?

Nó cho thấy lỗi lớn của truy xuất đa bước không chỉ nằm ở xếp hạng. Khi phương pháp biết khai thác tín hiệu title/entity để tìm tài liệu hỗ trợ thứ hai, `full_support@10` tăng từ 0.545 lên 0.620 trong pilot.

A Phụ lục cấu hình

Phụ lục này ghi các cấu hình đủ để mentor đối chiếu cách hệ thống đang chạy và cách các số trong báo cáo được sinh ra. Đây là cấu hình project/runtime hiện tại, không phải cấu hình production.

A.1 Hồ sơ dataset trong runtime

Mục	HotpotQA	VimQA
Dataset id	beir/hotpotqa/dev	vimqa/all
Index runtime	hotpotqa_full_bm25_current	vimqa_all_dense_bkai_current
Phương pháp	es_bm25, tv_dense, tv_hybrid, tv_filtered_hybrid	es_bm25, es_dense, es_hybrid
Mặc định	tv_hybrid	es_bm25
Backend dense	turbovec	elasticsearch_dense_vector
Model embedding	BAAI/bge-small-en-v1.5	bkai-foundation-models/vietnamese-bi-encoder
Số chiều vector	384	768
Metric chính	full_support_recall@10	recall@10
Metadata filter	Có	Có trong profile hiện tại

A.2 Cấu hình ingest và artifact

Hạng mục	Cấu hình
HotpotQA staging	artifacts/hotpotqa_full/staging, docs-per-file=50000, numeric_id ổn định.
HotpotQA document count	5,233,329 documents; validate bằng --expected-count 5233329.
HotpotQA BM25 index	Index vật lý hotpotqa_full_bm25_v1, alias runtime hotpotqa_full_bm25_current.
HotpotQA ingest batch	--batch-size 256, progress markers trong artifacts/hotpotqa_full/progress.
TurboVec artifact	artifacts/hotpotqa_full/turbovec/hotpotqa_bge_small_4bit.tvim.
TurboVec vector config	BGE-small, dim=384, bit_width=4.
VimQA staging	artifacts/vimqa/all/staging/docs-00000.jsonl, 3,623 tài liệu.
VimQA queries/qrels	evaluation/results/vimqa/vimqa_queries.tsv và vimqa_qrels.tsv, 9,044 truy vấn.
VimQA index	vimqa_all_bm25_current cho BM25 và vimqa_all_dense_bkai_current cho dense/hybrid.

A.3 Cấu hình retrieval và benchmark

Phương pháp	Cấu hình chính
es_bm25	Elasticsearch multi_match trên title^2 và content; top_k=10 trong benchmark chính.
tv_dense	Encode query bằng BGE-small qua embedding service, search TurboVec full index, hydrate tài liệu bằng Elasticsearch qua numeric_id.
tv_hybrid	BM25 candidates + TurboVec dense candidates, gộp bằng RRF; benchmark chính dùng candidate_k=100, num_candidates=100, rrf_k=30.
tv_filtered_hybrid	BM25/filter tạo allowlist numeric_id; TurboVec search trong allowlist rồi gộp RRF. Dùng cho metadata/filter path, không phải quality default.
tv_hybrid_rerank	Benchmark-only; rerank top-100 candidates bằng cross-encoder/ms-marco-MiniLM-L-6-v2; không expose làm runtime default.
tv_bridge_title_entities_rrf	Sinh truy vấn bước hai từ title/entity/lead-sentence; cấu hình khuyến nghị hiện tại là beam1_terms6: beam_size=1, max_bridge_terms=6.

Các benchmark HotpotQA trong báo cáo dùng cấu hình pilot:

```
dataset = beir/hotpotqa/dev
index = hotpotqa_full_bm25_current
top_k = 10
max_queries = 200
candidate_k = 100
num_candidates = 100
rrf_k = 30
first_hop_k = 5
second_hop_k = 10
context_chars = 256
```

Riêng cấu hình bridge tuning tốt nhất hiện tại:

```
method = tv_bridge_title_entities_rrf
config = beam1_terms6
beam_size = 1
max_bridge_terms = 6
top_k = 10
candidate_k = 100
num_candidates = 100
rrf_k = 30
```

A.4 Cấu hình runtime, API và cache

Biến/endpoint	Giá trị hoặc vai trò
DATASET_ID	Mặc định beir/hotpotqa/dev.
ELASTICSEARCH_URL	Mặc định local http://localhost:9200.
ELASTICSEARCH_INDEX	Local default hotpotqa_docs_current; Docker/demo dùng hotpotqa_full_bm25_current.
EMBEDDING_SERVICE_URL	Docker/demo dùng http://host.docker.internal:8010/embed.
EMBEDDING_TIMEOUT_SECONDS	30.
REDIS_URL	Docker dùng redis://redis:6379/0.
SEARCH_CACHE_TTL_SECONDS	Mặc định 300; demo warm cache có thể dùng 86400.
TURBOVEC_INDEX_PATH	Trong container: /app/artifacts/hotpotqa_full/turbovec/hotpotqa_bge_small_4bit.tvim.
DEFAULT_SEARCH_METHOD	tv_hybrid.

Các endpoint dataset-first chính:

```
GET /datasets
GET /datasets/{dataset_id}/stats
GET /datasets/{dataset_id}/queries
GET /datasets/{dataset_id}/benchmarks
GET /datasets/{dataset_id}/embedding-health
POST /datasets/{dataset_id}/search
```

A.5 Cấu hình semantic metadata

Mục	Cấu hình/ý nghĩa
Cờ bật parser	<code>semantic_metadata=true</code> ; nếu không bật, query đi theo đường retrieval bình thường.
Parser	Rule-based deterministic parser để tách <code>effective_query</code> và <code>metadata_filters</code> .
Trường filter hiện có	<code>author</code> , <code>created_at</code> , <code>modified_at</code> .
Ưu tiên filter	Filter nhập tay từ UI override filter parser sinh ra.
Embedding text	Không append metadata vào embedding text; vector chỉ biểu diễn content để tránh nhiễu semantic search.
Đường retrieval khi có metadata	<code>tv_hybrid</code> có filter được route sang <code>tv_filtered_hybrid</code> ; filter được xem là hard constraint.

A.6 Lệnh reproduce tối thiểu

```
python scripts/stage_hotpotqa.py --dataset beir/hotpotqa --output-dir
artifacts/hotpotqa_full/staging --docs-per-file 50000
python scripts/es_hotpotqa.py create-index --index hotpotqa_full_bm25_v1 --alias
hotpotqa_full_bm25_current --reset
python scripts/es_hotpotqa.py ingest --index hotpotqa_full_bm25_v1 --staging-dir
artifacts/hotpotqa_full/staging --progress-dir artifacts/hotpotqa_full/progress
--batch-size 256
python scripts/es_hotpotqa.py validate --index hotpotqa_full_bm25_current
--expected-count 5233329
```

```
python -m src.evaluation.benchmark_es --dataset beir/hotpotqa/dev --index
hotpotqa_full_bm25_current --methods es_bm25,tv_dense,tv_hybrid --top-k
10 --max-queries 200 --candidate-k 100 --num-candidates 100 --rrf-k 30
--output evaluation/results/hotpotqa_full/tv_full_200.json --run-dir
evaluation/runs/hotpotqa_full
```

A.7 Cấu hình render báo cáo

Báo cáo được render bằng MiKTeX/XeLaTeX để hỗ trợ tiếng Việt có dấu:

```
latexmk -pvc -view=none -xelatex -interaction=nonstopmode -halt-on-error
mentor-report.tex
```